



Databricks

INTERVIEWS

2026

What Actually Gets You Hired



SELECT - FROM candidates
WHERE skills = "Databricks"

Databricks Interview Questions

1. What is Databricks, and how does it differ from Apache Spark?

Answer: Databricks is a cloud-based platform that provides a collaborative environment for data engineers, data scientists, and analysts. It integrates seamlessly with Apache Spark, which is an open-source distributed computing system. Databricks enhances Spark's capabilities with additional features like an optimized runtime, advanced analytics tools, and a unified workspace for collaboration.

Example: While Apache Spark can be used on its own or through other platforms, Databricks provides a managed environment that includes built-in clusters, collaborative notebooks, and integration with Azure and AWS services. For instance, Databricks offers features like Delta Lake for improved data reliability and performance, which is not available in standard Apache Spark.

2. What is Delta Lake, and how does it improve data reliability?

Answer: Delta Lake is an open-source storage layer that brings reliability to data lakes. It provides ACID transactions, scalable metadata handling, and unifies streaming and batch data processing. Delta Lake allows for versioned data, which means you can easily time travel to previous versions of data and ensure data quality and consistency.

Example: Imagine you have a streaming application that continuously writes data to a data lake. With Delta Lake, you can ensure that the data is reliably stored, even if there are failures. If an update operation fails, Delta Lake's ACID transactions guarantee that the data remains consistent and correct.

3. How do you optimize Spark jobs in Databricks?

Answer: Optimizing Spark jobs involves several strategies:

- **Caching:** Store intermediate results in memory to avoid recomputation.
- **Partitioning:** Use proper data partitioning to reduce shuffle operations.

@Tech Interview Titans

- **Broadcast Joins:** Use broadcast joins for smaller datasets to reduce shuffle.
- **Delta Lake:** Leverage Delta Lake's optimization features like data skipping and Z-Ordering.

Example: If you have a large DataFrame that is used multiple times in your operations, you can cache it using `df.cache()`. This avoids recomputation and speeds up the job. Additionally, if you're joining a large DataFrame with a small one, using a broadcast join can reduce the amount of data shuffled between nodes.

4. Explain the concept of "Lazy Evaluation" in Spark.

Answer: Lazy evaluation in Spark means that transformations on data (like map and filter) are not executed immediately. Instead, Spark builds a logical plan of these transformations and only executes them when an action (like count or collect) is triggered. This approach optimizes the execution by reducing the number of passes over the data.

Example: If you have a sequence of transformations like `df.filter().select().groupBy()`, Spark will only execute these operations when an action like `df.show()` is called. This allows Spark to optimize the execution plan, possibly combining multiple operations into a single stage.

5. What are some common performance tuning techniques in Databricks?

Answer:

- **Adjust Cluster Configuration:** Choose appropriate cluster size and configuration based on workload requirements.
- **Optimize Data Storage:** Use Delta Lake for efficient data storage and management.
- **Data Skipping:** Utilize Delta Lake's data skipping capabilities to minimize I/O.
- **Parallelism:** Adjust the number of partitions and parallelism settings to match the cluster's capabilities.

@Tech Interview Titans

Example: If a job is running slowly, you might increase the number of nodes in your Databricks cluster to handle the load more efficiently. Additionally, using Delta Lake's Z-Ordering can help to reduce scan times by optimizing data layout.

6. Scenario: You have a large dataset in Azure Data Lake Storage (ADLS) and need to perform complex transformations. How would you approach this using Databricks?

Answer:

1. **Create a Databricks Cluster:** Set up a cluster with sufficient resources for your workload.
2. **Mount ADLS:** Use Databricks to mount ADLS to access the data.
3. **Read Data:** Use Spark to read the data from ADLS into a DataFrame.
4. **Transform Data:** Apply necessary transformations using Spark SQL or DataFrame API.
5. **Optimize Performance:** Utilize caching and partitioning to optimize the transformations.
6. **Write Results:** Save the transformed data back to ADLS or another data store using Delta Lake for better performance and reliability.

Example:

Mount ADLS

```
dbutils.fs.mount(source = "adls_path", mountPoint = "/mnt/adls",  
extraConfigs = {"<conf-key>":dbutils.secrets.get(scope = "<scope-name>",  
key = "<key-name>"))
```

Read Data

```
df = spark.read.format("csv").option("header",  
"true").load("/mnt/adls/large_dataset.csv")
```

@Tech Interview Titans

Transform Data

```
df_transformed = df.filter(df['column'] > 100).groupBy("category").agg({"value": "sum"})
```

Cache Data

```
df_transformed.cache()
```

Write Results

```
df_transformed.write.format("delta").save("/mnt/adls/transformed_data")
```

7. Scenario: You need to handle streaming data from multiple sources and perform real-time analytics. How would you implement this in Databricks?

Answer:

1. **Set Up a Streaming Cluster:** Create a Databricks cluster suitable for handling streaming workloads.
2. **Read Streaming Data:** Use Spark Structured Streaming to read data from sources like Kafka or Azure Event Hubs.
3. **Transform Data:** Apply real-time transformations and aggregations as the data arrives.
4. **Write to Sink:** Write the processed data to a sink, such as Delta Lake or a dashboard for real-time analytics.
5. **Monitor and Scale:** Monitor the performance and scale the cluster as needed.

Example:

Read Streaming Data

@Tech Interview Titans

```
streaming_df =  
spark.readStream.format("kafka").option("kafka.bootstrap.servers",  
"kafka_server:9092").option("subscribe", "topic_name").load()  
  
# Transform Data  
  
transformed_df = streaming_df.selectExpr("CAST(key AS STRING)",  
"CAST(value AS STRING)")  
  
# Write to Delta Lake  
  
query =  
transformed_df.writeStream.format("delta").outputMode("append").option("checkpointLocation",  
"/mnt/adls/checkpoints").start("/mnt/adls/streaming_data")  
  
# Await Termination  
  
query.awaitTermination()
```

8. What are the differences between batch and streaming data processing in Spark?

Answer:

- **Batch Processing:** Processes data in discrete chunks or batches. Suitable for scenarios where data is collected over time and analyzed periodically.
- **Streaming Processing:** Continuously processes data in real-time as it arrives. Suitable for scenarios requiring immediate analysis and response.

Example: Batch processing might be used for daily sales reports, where data is collected over a day and analyzed in one go. Streaming processing might be used for real-time fraud detection in financial transactions, where each transaction is analyzed as it arrives.

@Tech Interview Titans

9. Scenario: You need to merge two large datasets with different schemas using Databricks. How would you approach this?

Answer:

1. **Read Data:** Load both datasets into separate DataFrames.
2. **Schema Alignment:** Align schemas if necessary by renaming columns or adding missing columns.
3. **Merge Data:** Use Spark DataFrame operations to join the datasets.
4. **Handle Missing Data:** Address any missing or mismatched data appropriately.
5. **Optimize and Save:** Optimize the merged dataset and save it to a target location.

Example:

Read Data

```
df1 = spark.read.format("csv").option("header",  
"true").load("path_to_dataset1.csv")
```

```
df2 = spark.read.format("csv").option("header",  
"true").load("path_to_dataset2.csv")
```

Align Schemas

```
df2 = df2.withColumnRenamed("old_column_name",  
"new_column_name")
```

Merge Data

```
merged_df = df1.join(df2, df1["id"] == df2["id"], "outer")
```

Handle Missing Data

```
merged_df = merged_df.fillna("default_value")
```


@Tech Interview Titans

Save Results

```
merged_df.write.format("delta").save("path_to_output")
```

10. What are the key features of Databricks Runtime?

Answer: Databricks Runtime is an optimized version of Apache Spark provided by Databricks. Key features include:

- **Performance Enhancements:** Includes optimizations and performance improvements beyond open-source Spark.
- **Delta Lake Integration:** Provides support for Delta Lake, improving reliability and performance.
- **Runtime Libraries:** Comes pre-installed with various libraries and packages like MLlib, TensorFlow, and PyTorch.
- **Advanced Optimizations:** Includes features like Adaptive Query Execution (AQE) and dynamic resource allocation.

Example: Databricks Runtime can significantly speed up your Spark jobs by using optimizations like AQE, which adjusts the execution plan based on runtime statistics, leading to more efficient query execution.

11. Explain the concept of Adaptive Query Execution (AQE) in Databricks.

Answer: Adaptive Query Execution (AQE) is a feature in Databricks Runtime that dynamically optimizes query execution plans based on real-time statistics. AQE can adjust join strategies, optimize shuffle partitions, and improve query performance by reacting to actual data and runtime conditions.

Example: If a query involves a large shuffle operation, AQE can adjust the number of shuffle partitions based on the size of the data being shuffled, thereby improving performance and reducing overhead.

12. How can you manage data quality and integrity in Databricks?

@Tech Interview Titans

Answer: Data quality and integrity can be managed in Databricks using:

- **Delta Lake:** Provides ACID transactions, schema enforcement, and data versioning.
- **Data Validation:** Implement checks and validations using Spark transformations and actions.
- **Error Handling:** Use logging and monitoring to catch and handle errors in data processing pipelines.

Example: Using Delta Lake, you can enforce schema constraints to ensure that incoming data adheres to the expected format, reducing the chances of data quality issues.

13. Scenario: You need to handle a situation where a Spark job fails intermittently due to memory issues. How would you troubleshoot and resolve this in Databricks?

Answer:

1. **Check Logs:** Review Spark job logs to identify any memory-related errors or issues.
2. **Increase Memory:** Adjust the cluster configuration to provide more memory to the executors.
3. **Optimize Job:** Review and optimize Spark configurations such as executor memory settings and shuffle partition sizes.
4. **Data Partitioning:** Ensure that data is properly partitioned to avoid large shuffle operations that can lead to memory issues.
5. **Caching:** Use caching judiciously to avoid recomputing intermediate results that consume additional memory.

Example: If the logs indicate an `OutOfMemoryError`, increasing the executor memory in the cluster configuration and reviewing the job's data partitioning strategy can help mitigate the issue.

14. How do you perform data governance in Databricks?

@Tech Interview Titans

Answer: Data governance in Databricks involves:

- **Access Controls:** Use Databricks' role-based access control (RBAC) to manage user permissions and access to data.
- **Data Lineage:** Track the lineage of data transformations and ensure traceability using tools integrated with Databricks.
- **Auditing:** Implement logging and monitoring to track data access and changes.
- **Compliance:** Ensure data compliance with regulations by using features like data encryption and audit logs.

Example: You can set up access controls in Databricks by assigning specific roles to users, such as admin, editor, or viewer, ensuring that only authorized users can access or modify sensitive data.

15. What are the differences between Databricks Notebooks and Jupyter Notebooks?

Answer:

- **Databricks Notebooks:** Integrated with Databricks' collaborative workspace, supports Spark-based computations, and includes features like version control, job scheduling, and interactive visualizations.
- **Jupyter Notebooks:** Open-source and more general-purpose, supports multiple languages (Python, R, Julia), and is typically used for local or standalone development.

Example: Databricks Notebooks offer seamless integration with Databricks clusters, allowing users to run distributed Spark jobs directly within the notebook, while Jupyter Notebooks might require additional setup to connect to a Spark cluster.

@Tech Interview Titans

16. Scenario: You are working with a large dataset that needs to be processed in batches. How would you implement batch processing in Databricks?

Answer:

1. **Create a Databricks Cluster:** Set up a cluster with appropriate resources for batch processing.
2. **Load Data:** Use Spark to load data from the source (e.g., ADLS, S3).
3. **Batch Processing:** Apply transformations and actions on the data in batches using Spark DataFrame or SQL operations.
4. **Save Results:** Write the processed data to a storage location in a format like Parquet or Delta Lake.
5. **Job Scheduling:** Use Databricks jobs to schedule and automate the batch processing.

Example:

```
df = spark.read.format("parquet").load("path_to_large_dataset")
```

```
# Batch Processing
```

```
batch_df = df.filter(df['date'] >= '2024-01-01').groupBy("category").agg({"value": "sum"})
```

```
# Save Results
```

```
batch_df.write.format("delta").mode("append").save("path_to_output")
```

17. What are some best practices for managing Spark configurations in Databricks?

Answer:

- **Tune Executor and Driver Memory:** Configure executor and driver memory based on the workload and dataset size.

@Tech Interview Titans

- **Adjust Spark Properties:** Set properties such as `spark.sql.shuffle.partitions` to optimize performance based on the data size.
- **Monitor and Optimize:** Use Databricks' built-in monitoring tools to analyze performance and make necessary adjustments.
- **Use Dynamic Allocation:** Enable dynamic allocation to automatically adjust the number of executors based on the workload.

Example: You might adjust the `spark.sql.shuffle.partitions` setting based on the size of the data being shuffled to reduce the overhead of shuffle operations.

18. Scenario: You need to perform machine learning on a large dataset using Databricks. How would you set up and run the ML workflows?

Answer:

1. **Prepare Data:** Load and preprocess the data using Spark DataFrame operations.
2. **Feature Engineering:** Use Spark MLlib or Databricks' ML runtime for feature engineering.
3. **Train Model:** Utilize MLlib, TensorFlow, or PyTorch for model training on the prepared data.
4. **Evaluate Model:** Evaluate the model using appropriate metrics and validation techniques.
5. **Deploy Model:** Save and deploy the model for inference or use Databricks' MLflow for tracking and managing experiments.

Example:

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Prepare Data
```


@Tech Interview Titans

```
df = spark.read.format("csv").option("header",  
"true").load("path_to_data")
```

```
# Train Model
```

```
lr = LogisticRegression(featuresCol="features", labelCol="label")  
model = lr.fit(df)
```

```
# Evaluate Model
```

```
evaluator = BinaryClassificationEvaluator(labelCol="label")  
accuracy = evaluator.evaluate(model.transform(df))
```

```
print("Model Accuracy:", accuracy)
```

19. How does Databricks handle job orchestration and scheduling?

Answer: Databricks provides job orchestration and scheduling through:

- **Databricks Jobs:** Allows users to schedule notebooks, JARs, or Python scripts to run on clusters at specified intervals or triggered by events.
- **Job Clusters:** Creates clusters specifically for running jobs, which can be terminated after the job completes to save costs.
- **Task Dependencies:** Supports setting up task dependencies within jobs to manage complex workflows.

Example: You can create a job in Databricks that schedules a notebook to run daily at a specific time and sets up tasks to run sequentially or in parallel.

20. Scenario: You have multiple teams working on different projects in Databricks. How would you manage collaboration and data sharing among these teams?

@Tech Interview Titans

Answer:

1. **Workspace Organization:** Organize workspaces and folders for different teams or projects.
2. **Access Control:** Set up role-based access controls (RBAC) to manage permissions for different teams.
3. **Shared Notebooks:** Share notebooks and data with specific teams or users using Databricks' sharing features.
4. **Data Catalog:** Use a data catalog or metadata management tool to facilitate data discovery and governance.

Example: You can create separate folders for each team within the Databricks workspace and assign permissions so that only authorized team members can access and modify the data and notebooks.

1. What is the architecture of Databricks, and how does it integrate with Apache Spark?

Answer: Databricks architecture consists of the following components:

- **Databricks Workspace:** Provides a collaborative environment where users can create and manage notebooks, libraries, and jobs.
- **Databricks Runtime:** An optimized version of Apache Spark that includes performance enhancements and additional features like Delta Lake.
- **Cluster Manager:** Manages and provisions Spark clusters for executing jobs and interactive workloads.
- **Job Scheduler:** Allows users to schedule and automate jobs and tasks.
- **Data Integration:** Integrates with cloud storage solutions like Azure Data Lake Storage (ADLS) and Amazon S3, enabling seamless data access and management.

Integration with Apache Spark: Databricks enhances Apache Spark by providing an optimized runtime environment that includes performance improvements, additional libraries, and seamless integration with cloud data stores. It abstracts much of the complexity involved in managing and tuning

@Tech Interview Titans

Spark clusters, making it easier to run Spark jobs and collaborate on data projects.

2. Explain the concept of Delta Lake and its benefits.

Answer: Delta Lake is an open-source storage layer that brings ACID transactions to data lakes. It is designed to address some of the limitations of traditional data lakes by providing:

- **ACID Transactions:** Ensures data reliability and consistency by supporting atomicity, consistency, isolation, and durability.
- **Schema Evolution:** Allows changes to the schema of the data without disrupting the existing data pipeline.
- **Time Travel:** Provides the ability to query historical versions of data, which is useful for auditing and debugging.
- **Unified Batch and Streaming:** Enables seamless integration of batch and streaming data processing using the same APIs.

Benefits:

- **Improved Data Reliability:** Guarantees that data operations are consistent and durable.
 - **Enhanced Performance:** Optimizes data storage and query performance through features like data skipping and Z-Ordering.
 - **Simplified Data Management:** Makes it easier to handle schema changes and data versioning.
-

3. What is the role of the Databricks Runtime in optimizing Spark workloads?

Answer: Databricks Runtime is a managed Spark environment that includes various optimizations and enhancements to improve the performance of Spark workloads. It includes:

- **Performance Optimizations:** Includes features such as Adaptive Query Execution (AQE) and dynamic resource allocation to optimize job performance.

@Tech Interview Titans

- **Enhanced Libraries:** Comes with pre-installed libraries and tools, such as Delta Lake and MLlib, which are optimized for better performance.
- **Cluster Management:** Automates cluster provisioning and management, reducing the overhead of managing Spark clusters.

The Databricks Runtime helps in optimizing Spark workloads by providing a more efficient execution environment and advanced features that enhance performance and reliability.

4. How does Databricks handle security and compliance?

Answer: Databricks addresses security and compliance through several mechanisms:

- **Role-Based Access Control (RBAC):** Allows administrators to define fine-grained access permissions for users and groups.
- **Data Encryption:** Supports encryption of data at rest and in transit to protect sensitive information.
- **Audit Logs:** Provides detailed logs of user activities and data access, helping with compliance and monitoring.
- **Compliance Certifications:** Databricks complies with various industry standards and regulations, such as GDPR, HIPAA, and SOC 2.

These features ensure that data is protected and that organizations can meet their security and compliance requirements.

5. What are the key differences between Databricks and traditional data processing platforms?

Answer:

- **Managed Environment:** Databricks provides a fully managed environment with automated cluster provisioning and management, whereas traditional platforms often require manual setup and maintenance.
- **Optimization:** Databricks offers an optimized runtime with performance enhancements and additional features like Delta Lake, which may not be available in traditional platforms.